JOHANNES KEPLER
UNIVERSITY LINZ

Author
Lukas Bauer
k12317693

Submission
Institute of Financial Mathematics and Applied Number Theory

Thesis Supervisor
Assoz. Uniprof. Dr.
Sascha Desmettre

June 2026

Reinforcement Learning for Portfolio Optimization: A Comparison with the Markowitz Approach



Bachelor's Thesis

to confer the academic degree of

Bachelor of Science

in the Bachelor's Program

Technische Mathematik

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract

The study of reinforcement learning for portfolio optimization has gained traction in recent years. While traditional methods such as the mean-variance approach introduced by Harry Markowitz provide a stable and widely recognized baseline for portfolio optimization, they can struggle in real markets due to their relatively static nature and reliance on historical estimates. Reinforcement learning therefore stands out as a promising research direction, as it can develop adaptive strategies through continuous learning, even in complex and changing market regimes.

This thesis investigates how the Proximal Policy Optimization (PPO) algorithm compares to the classic Markowitz approach under daily and monthly rebalancing, both with and without transaction costs. The analysis is based on large-cap U.S. equities and evaluates the strategies across two market regimes, including a stressed market environment covering the 2008 financial crisis. This allows the thesis to examine whether the reinforcement learning agent is able to adapt not only during more recent market conditions, but also during periods of substantial financial stress.

The results show that reinforcement learning can achieve competitive risk-adjusted returns, but that its performance strongly depends on the rebalancing frequency. While PPO shows promising results in the monthly rebalancing setting, its performance deteriorates under daily trading once transaction costs are included, mainly due to the high turnover generated by the reinforcement learning strategies. These findings suggest that the reinforcement learning agent is not definitively superior to the Markowitz model. Rather, its usefulness depends heavily on the trading frequency and the market environment in which it is applied.

Contents

Abstract ii

1 Literature Review 1

2 Modern Portfolio Theory 2
- 2.1 Introduction . . . 2
- 2.2 Basics of Markowitz Optimization . . . 2
- 2.3 Parameter Estimation in the Markowitz Framework . . . 4
- 2.4 Portfolio Performance Metrics . . . 6
  - 2.4.1 Cumulative and Annualized Return . . . 7
  - 2.4.2 Maximum Drawdown . . . 7
  - 2.4.3 Volatility . . . 7
  - 2.4.4 Sharpe Ratio . . . 8

3 Reinforcement Learning 9
- 3.1 Introduction . . . 9
- 3.2 Formalization as a Markov Decision Process . . . 9
- 3.3 Return and Discounted Reward . . . 11
- 3.4 Policies and Value Functions . . . 11
- 3.5 Value-Based Reinforcement Learning Algorithms . . . 12
  - 3.5.1 Monte Carlo Methods . . . 13
  - 3.5.2 Temporal-Difference Learning . . . 14
- 3.6 Policy-Based Reinforcement Learning Algorithms . . . 15
  - 3.6.1 REINFORCE . . . 16
  - 3.6.2 Generalized Advantage Estimation . . . 17
  - 3.6.3 Proximal Policy Optimization . . . 18

4 Experimental Setup 21
- 4.1 Introduction . . . 21
- 4.2 Data and Asset Universe . . . 21
- 4.3 General Experimental Framework . . . 22
- 4.4 Markowitz Setup . . . 23
- 4.5 Reinforcement Learning Setup . . . 23
- 4.6 Evaluation . . . 24

5 Results 25
- 5.1 Results for the 2019 - 2026 Market Regime . . . 25
  - 5.1.1 Daily Rebalancing without Transaction Costs . . . 25
  - 5.1.2 Daily Rebalancing with Transaction Costs . . . 26
  - 5.1.3 Monthly Rebalancing without Transaction Costs . . . 27
  - 5.1.4 Monthly Rebalancing with Transaction Costs . . . 29
- 5.2 Results for the 1998-2015 Market Regime . . . 30
  - 5.2.1 Monthly Rebalancing with Transaction Costs . . . 30

6 Limitations 33

7 Conclusion and Outlook **34**

Bibliography **35**

Appendix A Full Asset Universe List and PPO Hyperparameter Specification **38**

Appendix B 2007 - 2015 Regime Data **40**
B.1 Monthly Rebalancing without Transaction Costs 40
B.2 Daily Rebalancing with Transaction Costs 41
B.3 Daily Rebalancing without Transaction Costs 42

Chapter 1

Literature Review

Portfolio optimization has been studied for decades, with the mean-variance framework of Markowitz forming one of the central foundations of modern portfolio theory [12]. Subsequent work has improved the empirical stability of this approach, for example through shrinkage estimators such as the Ledoit-Wolf covariance estimator [8]. These developments make Markowitz optimization a natural and interpretable benchmark for evaluating newer portfolio allocation methods.

In recent years, reinforcement learning has increasingly been applied to portfolio optimization. Instead of solving a static allocation problem, reinforcement learning treats portfolio management as a sequential decision problem in which an agent repeatedly observes market information, chooses portfolio weights and receives a reward based on the resulting portfolio performance. Empirical studies such as Durall [4] and Lim et al. [10] show that reinforcement learning can be used for dynamic asset allocation and portfolio rebalancing, but also that performance depends strongly on the chosen environment, reward structure and rebalancing mechanism.

More recent work has therefore focused less on whether reinforcement learning can be applied to portfolio management in general and more on how such methods can be made more practical. Zhang et al. [29] address this issue by proposing a cost-sensitive reinforcement learning framework that explicitly incorporates transaction and risk costs into the reward function. Similarly, DeepAries [6] focuses on the rebalancing-frequency problem by allowing the agent to choose between different holding intervals rather than trading at a fixed frequency. Its results are particularly relevant for this thesis, since fixed monthly rebalancing is shown to be a strong baseline compared with fixed daily rebalancing and, in some metrics, performs competitively even relative to the adaptive strategy.

Building on this literature, the present thesis does not propose a new reinforcement learning architecture. Instead, it provides a controlled empirical comparison between a PPO-based reinforcement learning agent, Markowitz optimization and an equal-weight benchmark. The analysis compares fixed daily and fixed monthly rebalancing, both with and without transaction costs, across two different U.S. equity market regimes. In contrast to approaches that approximate monthly rebalancing by a fixed number of trading days, the monthly strategy used here follows a calendar-based convention. This allows the thesis to examine how the practical choice of rebalancing frequency affects the relative performance of reinforcement learning and classical portfolio optimization.

Chapter 2

Modern Portfolio Theory

2.1 Introduction

Modern Portfolio Theory (MPT) originates from Harry Markowitz's 1952 paper on portfolio selection and aims to identify an optimal investment strategy [12]. At its core, the theory formalizes the idea that investors are risk-averse: among portfolios with the same expected return, they prefer the one with lower risk. This insight laid the foundation for mean-variance analysis and has had a lasting influence on portfolio management and financial economics.

The mean-variance approach is considered a one-period model, meaning that investment decisions are made only at the beginning of the investment horizon and outcomes are observed at the end, without any further action in between. While this simplicity may lead to disadvantages compared to continuous-time approaches [13], the Markowitz model still maintains its relevance.

This chapter introduces the basic concepts of Modern Portfolio Theory, discusses estimation methods for the quantities required in the mean-variance framework and presents portfolio evaluation measures used later in the comparison with the reinforcement learning approach.

2.2 Basics of Markowitz Optimization

The notation used in this chapter follows that of Korn [7].

Consider a market where d different securities with prices $s_1, s_2, \dots, s_d > 0$ at initial time $t = 0$ are traded. The security prices $S_1(T), S_2(T), \dots, S_d(T)$ at terminal time $t = T$ are not known and are therefore modeled as non-negative random variables on a probability space $(\Omega, \mathcal{F}, \mathbb{P})$.

The return of each security is then defined as

$$R_i(T) = \frac{S_i(T) - s_i}{s_i}, \qquad i = 1, \dots, d. \tag{2.1}$$

Throughout this chapter, the means and covariances of asset returns are assumed to be known:

$$\mathbb{E}[R_i(T)] = \mu_i, \qquad i = 1, \dots, d, \tag{2.2}$$

$$\mathrm{Cov}[R_i(T), R_j(T)] = \sigma_{ij}, \qquad i, j = 1, \dots, d. \tag{2.3}$$

In practice, these quantities are unknown and must be estimated from data, as discussed later in this chapter. It is further assumed that an investor may hold

$\varphi_i \in \mathbb{R}$ shares of asset i, with $\varphi_i \geqslant 0$ for $i = 1, \dots, d$. Thus, short-selling is not allowed. In addition, transaction costs are neglected in the basic formulation considered here. This corresponds to the standard classical Markowitz framework and provides the baseline for the subsequent analysis.

Definition 2.1. *Given an investor with budget $x > 0$ and holdings $\varphi_i \geqslant 0$ such that*

$$\sum_{i=1}^{d} \varphi_i \cdot s_i = x \tag{2.4}$$

the portfolio vector $\pi = (\pi_1, \pi_2, \dots, \pi_d) \in \mathbb{R}^d$ is defined as

$$\pi_i = \frac{\varphi_i \cdot s_i}{x} \tag{2.5}$$

and the corresponding portfolio return is defined as

$$R^\pi = \sum_{i=1}^{d} \pi_i \cdot R_i(T). \tag{2.6}$$

The components of the portfolio vector, often called portfolio weights, represent the fractions of the total budget x invested in each security. By definition, the portfolio weights satisfy

$$\sum_{i=1}^{d} \pi_i = \frac{\sum_{i=1}^{d} \varphi_i \cdot s_i}{x} = \frac{x}{x} = 1,$$

so the full budget is invested.

The mean and variance of the portfolio return are then given by

$$\mathbb{E}[R^\pi] = \sum_{i=1}^{d} \pi_i \mu_i = \mu' \pi \tag{2.7}$$

and

$$\mathrm{Var}[R^\pi] = \sum_{i=1}^{d} \sum_{j=1}^{d} \pi_i \sigma_{ij} \pi_j = \pi' \sigma \pi. \tag{2.8}$$

Here μ' and π' denote the transpose of μ and π, respectively and σ is the covariance matrix with entries σ_{ij}.

The investor aims to achieve a favorable trade-off between expected return and risk. If only expected return is taken into account, the resulting portfolio will typically allocate the entire budget to the security with the highest expected return. In practice, such a concentrated investment strategy is highly risky. Markowitz therefore introduces portfolio variance as a measure of risk to capture the trade-off between return and uncertainty.

There are several equivalent ways to formulate this trade-off as an optimization problem. One formulation is to require a minimum expected return and, among all portfolios satisfying this requirement, choose the one with minimal variance:

$$\begin{aligned} \min_{\pi \in \mathbb{R}^d} \quad & \mathrm{Var}(R^\pi) \\ \text{subject to} \quad & \pi_i \geqslant 0, \quad i = 1, \dots, d, \\ & \sum_{i=1}^{d} \pi_i = 1, \\ & \mathbb{E}[R^\pi] \geqslant c_1. \end{aligned} \tag{2.9}$$

Alternatively, one may set an upper bound on the variance and then choose, among the remaining portfolios, the one with the highest expected return:

$$\begin{aligned} \max_{\pi \in \mathbb{R}^d} \quad & \mathbb{E}[R^\pi] \\ \text{subject to} \quad & \pi_i \geqslant 0, \quad i = 1, \dots, d, \\ & \sum_{i=1}^{d} \pi_i = 1, \\ & \mathrm{Var}(R^\pi) \leqslant c_2. \end{aligned} \tag{2.10}$$

In practice, a third equivalent formulation combines expected return and variance into a single objective function:

$$\mu' \pi - \frac{1}{2} \lambda \pi' \sigma \pi \tag{2.11}$$

which is maximized subject to the long-only and fully invested constraints. Here, $\lambda > 0$ is a risk-aversion parameter. Larger values of λ place more weight on variance reduction and therefore lead to more conservative portfolios, while smaller values emphasize expected returns more strongly.

In general, these optimization problems do not produce a single optimal portfolio, but a family of portfolios that depends on the chosen parameter c_1, c_2, or λ. This leads to the notion of efficiency.

A portfolio is called efficient if there exists no other admissible portfolio that achieves a strictly lower variance without reducing expected return, or a strictly higher expected return without increasing variance. The set of all such portfolios is called the efficient frontier [12].

The efficient frontier is usually represented in the plane spanned by expected return and the standard deviation of portfolio return. Although the optimization problems are formulated in terms of variance, graphical representations usually use standard deviation. Unlike variance, standard deviation is expressed in the same units as returns and is therefore easier to interpret.

Figure 2.1 shows 5,000 randomly generated long-only portfolios for a portfolio universe consisting of Amazon, Microsoft and Apple, together with the corresponding efficient frontier. The simulated portfolios illustrate the feasible set, whereas only portfolios on the frontier are efficient.

2.3 Parameter Estimation in the Markowitz Framework

The optimization problems formulated so far rely on the assumption that the expected returns and covariances of the individual securities are known. However, in practice, this assumption is unrealistic and the required quantities must be estimated from data. Such estimates are inherently uncertain, since past observations do not necessarily describe future market behavior well. In particular, estimating the expected return vector μ is notoriously difficult. Even small changes in estimated expected returns can lead to large changes in the optimal portfolio weights because the Markowitz optimizer tends to allocate heavily to assets with the highest estimated returns. These are often precisely

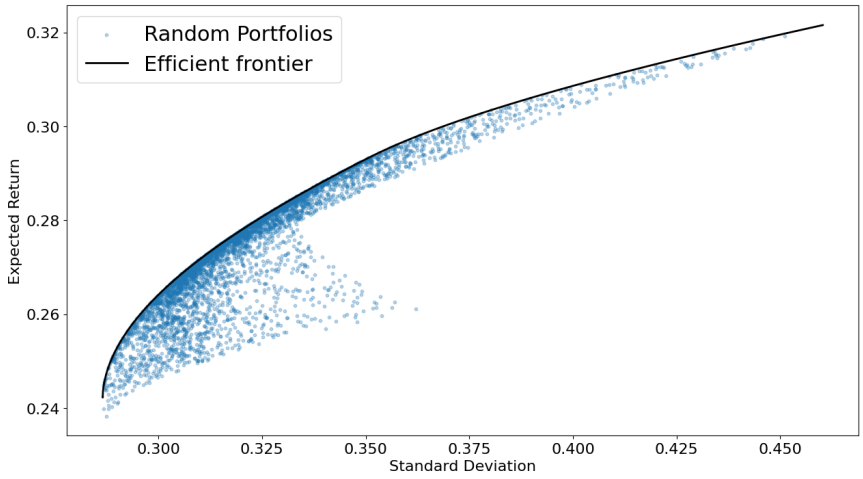


Figure 2.1: Feasible set of portfolios and the efficient frontier in the mean-standard deviation plane.

the ones most affected by sampling error. For this reason, mean-variance optimization is often described as an instance of "error maximization" [15].

Despite these difficulties, a common benchmark is to estimate μ using the historical sample mean of asset returns. Suppose that for each asset $i = 1, \dots, d$, historical return observations $R_{i,1}, \dots, R_{i,T}$ over T periods are available. Then the expected return of asset i can be estimated by

$$\hat{\mu}_i = \frac{1}{T} \sum_{t=1}^{T} R_{i,t}. \tag{2.12}$$

Even though this estimator is conceptually simple and easy to implement, it can be sensitive to the choice of sample period and may therefore produce unstable estimates. Nevertheless, it is used in this thesis as the reference approach within the Markowitz framework. More structured approaches to estimating expected returns have also been proposed, including the Capital Asset Pricing Model (CAPM) [11, 16, 22] and the Black-Litterman model [3]. Since neither of these approaches is used in this thesis, they are not discussed further.

Besides the expected return vector μ, mean-variance optimization also requires an estimate of the covariance matrix σ. Although covariance estimation is not without difficulties, it is generally considered less problematic than estimating expected returns, since variances and covariances can typically be estimated more reliably from historical return data [14]. A natural benchmark is therefore the sample covariance matrix:

$$\hat{\sigma}_{ij} = \frac{1}{T-1} \sum_{t=1}^{T} \left(R_{i,t} - \hat{\mu}_i\right)\left(R_{j,t} - \hat{\mu}_j\right), \qquad i, j = 1, \dots, d. \tag{2.13}$$

Although the sample covariance matrix is a simple and unbiased baseline, it can be unstable in practice, especially when the number of assets is large relative to the available sample size. For this reason, more robust covariance estimators

have been proposed. One prominent example is the Ledoit-Wolf shrinkage estimator [8], which combines the sample covariance matrix with a structured target matrix. The idea is to reduce estimation noise by shrinking the sample covariance matrix toward a more stable object. More intuitively, shrinkage dampens extreme values in the sample covariance matrix by moving them toward a more regular structure, since unusually high or low estimated covariances are often partly driven by estimation noise. In general, it has the form

$$\hat{\sigma}^{LW} = \delta F + (1 - \delta)\hat{\sigma}, \tag{2.14}$$

where $\hat{\sigma}$ denotes the sample covariance matrix, F is a shrinkage target and $\delta \in [0, 1]$ is the shrinkage intensity.

In the constant-correlation target considered here, the target matrix $F = (f_{ij})_{i,j=1,\dots,d}$ is defined by

$$f_{ii} = \hat{\sigma}_{ii}, \qquad i = 1, \dots, d, \tag{2.15}$$

and

$$f_{ij} = \bar{r}\sqrt{\hat{\sigma}_{ii}\hat{\sigma}_{jj}}, \qquad i \neq j, \tag{2.16}$$

where

$$\bar{r} = \frac{2}{d(d-1)} \sum_{i=1}^{d-1} \sum_{j=i+1}^{d} r_{ij} \tag{2.17}$$

denotes the average sample correlation and

$$r_{ij} = \frac{\hat{\sigma}_{ij}}{\sqrt{\hat{\sigma}_{ii}\hat{\sigma}_{jj}}} \tag{2.18}$$

is the sample correlation between assets i and j.

To motivate the form of the target matrix, note that each covariance can be written as the product of the corresponding standard deviations and the correlation between the two assets. Thus, changing the correlation term also changes the covariance. In the constant-correlation target, the individual sample correlations are therefore replaced by their average, which regularizes the covariance matrix. As a result, the influence of extreme or noisy covariance estimates is reduced, which can improve the stability of the resulting portfolio optimization.

The shrinkage intensity δ can in principle take any value in $[0, 1]$. Ledoit and Wolf [8] derive an optimal value δ^* that minimizes the expected distance between the shrinkage estimator and the true covariance matrix. This estimator is used in the Markowitz implementation in the later chapters of this thesis.

2.4 Portfolio Performance Metrics

While the efficient frontier is a useful visual tool for identifying and understanding optimal portfolios, it is less suitable for comparing the performance of different portfolios. To assess portfolio performance, the following measures are introduced and later used to compare the Markowitz portfolio with the reinforcement learning portfolio.

2.4.1 Cumulative and Annualized Return

While the Markowitz framework is formulated as a one-period model, portfolio performance is typically assessed over multiple periods. For this purpose, consider a sequence of portfolio returns R_t^π, $t = 1, \dots, T$. Let V_0 denote the initial portfolio value. The portfolio value at time t is then given by

$$V_t = V_{t-1}(1 + R_t^\pi), \qquad t = 1, \dots, T. \tag{2.19}$$

Furthermore, the cumulative return over the period $[0, T]$ is defined as

$$\text{Cumulative Return} = \frac{V_T - V_0}{V_0}. \tag{2.20}$$

Closely related is the annualized return, also referred to as the Compound Annual Growth Rate (CAGR), which expresses the average yearly growth rate over the investment horizon:

$$\text{CAGR} = \left(\frac{V_T}{V_0}\right)^{\frac{1}{n}} - 1 \tag{2.21}$$

where n is the number of years considered. For daily data, a common choice is $n = \frac{T}{252}$, where 252 is an approximation of the number of trading days in a year. While cumulative returns express total wealth creation, annualized returns allow for comparisons across investment periods of different lengths.

2.4.2 Maximum Drawdown

The maximum drawdown (MDD) quantifies the largest observed decline in portfolio value from a previous peak over a given time period. More formally, it can be written as

$$\text{MDD} = \max_{t=0,\dots,T} \left(\frac{\max_{\tau=0,\dots,t} V_\tau - V_t}{\max_{\tau=0,\dots,t} V_\tau} \right). \tag{2.22}$$

The MDD serves as a measure of downside risk, particularly for risk-averse investors looking to limit substantial losses.

2.4.3 Volatility

In theoretical terms, volatility is defined as the standard deviation of portfolio returns:

$$\text{Volatility} = \sqrt{\text{Var}[R^\pi]}. \tag{2.23}$$

In the empirical evaluation, this quantity is estimated from the realized return series. It represents the degree of fluctuation in portfolio returns. Higher volatility implies greater uncertainty and risk, whereas lower volatility signals more stable returns.

2.4.4 Sharpe Ratio

Building on return and volatility, the Sharpe ratio [23] measures the excess return of a portfolio per unit of risk:

$$S = \frac{\mathbb{E}[R^\pi] - r_f}{\sqrt{\mathrm{Var}(R^\pi)}}. \tag{2.24}$$

Here r_f is the risk-free rate, which is the theoretical return of an investment with zero risk. The Sharpe ratio provides a measure of risk-adjusted return. A higher Sharpe ratio indicates a more efficient portfolio, as it achieves higher excess returns for the same level of risk.

In practical implementations, both the expected return and standard deviation are replaced by the sample mean and sample standard deviation, respectively.

In summary, this chapter presented the foundations of the mean-variance approach required for the remainder of this thesis. The Markowitz portfolio optimization problem was introduced and the concept of the efficient frontier was explained. Moreover, methods for estimating the parameters μ and σ were discussed, along with the performance measures that will later be used to evaluate the portfolios. While the Markowitz framework provides an intuitive and tractable baseline for portfolio optimization, its sensitivity to parameter estimation and its one-period structure with infrequent rebalancing motivate the study of sequential decision-making approaches. The next chapter therefore turns to reinforcement learning.

Chapter 3

Reinforcement Learning

3.1 Introduction

Reinforcement learning is a computational framework for sequential decision-making, in which an agent learns through interaction with an environment. At each time step, the agent observes the current state, selects an action and receives feedback from the environment in the form of a numerical reward [25]. The objective is to learn a strategy, also called a policy, that maximizes the expected cumulative reward over time.

The reinforcement learning problem is formalized as a Markov decision process (MDP), which consists of a state space, an action space, transition dynamics and a reward function [1, 5]. On the basis of this formulation, several classes of reinforcement learning methods have been developed, including Monte Carlo methods, temporal-difference methods [27] and policy optimization techniques [21, 28]. These methods can be categorized as model-based or model-free depending on whether a model of the environment's dynamics is available.

This chapter introduces the theoretical foundations of reinforcement learning needed in later chapters. Although the portfolio optimization model studied in this thesis is ultimately approached using policy-based methods, value-based methods are briefly reviewed because their ideas, especially bootstrapping and temporal-difference learning, are important for actor-critic methods and generalized advantage estimation.

3.2 Formalization as a Markov Decision Process

The notation used in the following chapters follows that of *Reinforcement Learning: An Introduction* [25].

As introduced in section 3.1, at each discrete time step $t = 0, 1, 2, \ldots$ the agent interacts with the environment. The agent observes the current state $S_t \in \mathcal{S}$ and selects an action $A_t \in \mathcal{A}$, where $\mathcal{S}$ and $\mathcal{A}$ denote the set of all possible states and actions, respectively. One time step later, the agent receives a numerical reward signal $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ and finds itself in a new state S_{t+1}.

Such a cycle of interactions generates a sequence of states, actions and rewards of the form

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \ldots \tag{3.1}$$

This is called a trajectory.

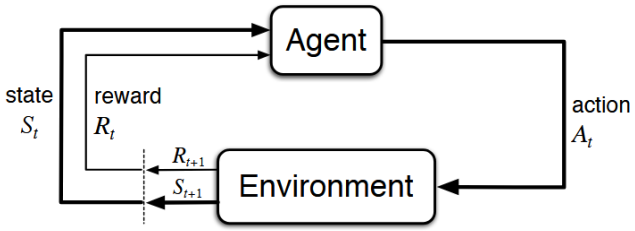


Figure 3.1: The agent-environment interaction in a Markov decision process [25].

Definition 3.1. *The dynamics of the MDP are described by the transition-reward probability function*

$$p(s', r \mid s, a) = \mathbb{P}(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a). \tag{3.2}$$

This function specifies the probability of receiving reward r when transitioning to state s' after taking action a in state s. This formulation assumes the Markov property. Informally, this means that the next state and reward depend only on the current state and action, not on the full past history. In other words, the state S_t is assumed to summarize all information needed to predict the future. Formally, this can be written as

$$\mathbb{P}(S_{t+1}, R_{t+1} \mid S_0, A_0, R_1, \dots, S_t, A_t) = \mathbb{P}(S_{t+1}, R_{t+1} \mid S_t, A_t). \tag{3.3}$$

Based on this distribution, the expected immediate reward can be expressed by the reward function

$$r(s, a) = \mathbb{E}[R_{t+1} \mid S_t = s, A_t = a]. \tag{3.4}$$

Using these components, the reinforcement learning problem can be formalized as a Markov decision process, following the formulation in *Markov Decision Processes: Discrete Stochastic Dynamic Programming* [18].

Definition 3.2 (Markov Decision Process). *A Markov decision process (MDP) is a tuple*

$$(\mathcal{S}, \mathcal{A}, p, r)$$

where

- $\mathcal{S}$ *is the set of states,*
- $\mathcal{A}$ *is the set of actions,*
- $p(s', r \mid s, a)$ *is the transition-reward probability function,*
- $r(s, a)$ *is the expected reward function.*

For each state-action pair (s, a)*, the probabilities satisfy*

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r \mid s, a) = 1.$$

3.3 Return and Discounted Reward

The objective of the reinforcement learning agent is to maximize the cumulative reward obtained over time. The cumulative reward starting from time step t is denoted by G_t, which is also called the return. Given a sequence of rewards $R_{t+1}, R_{t+2}, \ldots$, the simplest case is to define

$$G_t = R_{t+1} + R_{t+2} + \ldots + R_T,$$

where T is a terminal time step. This definition becomes problematic if the time horizon is infinite, since the sum $\sum_{k=t+1}^{\infty} R_k$ may diverge.

Therefore, the more common approach is to define G_t as the discounted reward

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1},$$

where $\gamma \in [0, 1]$ is the discount factor and a larger γ corresponds to more importance being placed on future rewards. In this definition, T can also be infinite; however, this requires $\gamma \in [0, 1)$ to ensure convergence. Additionally, the definition provides a recursive formula for computing the return:

$$\begin{aligned} G_t &\doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \ldots \\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \ldots) \\ &= R_{t+1} + \gamma G_{t+1}. \end{aligned} \tag{3.5}$$

Based on the definition of the return, the value of states and actions can be defined in terms of expected return, as discussed in the following section.

3.4 Policies and Value Functions

Intuitively, value functions provide a measure of how "good" it is for the agent to be in a particular state. This notion of goodness is quantified in terms of the expected return obtained when the agent starts in that state and follows a particular strategy. The strategy that determines how the agent selects actions is called a policy and is denoted by π.

A policy can be either deterministic or stochastic. A deterministic policy is a mapping

$$\pi : \mathcal{S} \to \mathcal{A},$$

which assigns to each state $s \in \mathcal{S}$ a specific action $\pi(s) \in \mathcal{A}$. In contrast, a stochastic policy specifies a probability distribution over actions for each state and is described by

$$\pi(a \mid s) = \mathbb{P}(A_t = a \mid S_t = s),$$

which denotes the probability of selecting action a when the agent is in state s. For every state $s \in \mathcal{S}$, these probabilities satisfy

$$\sum_{a \in \mathcal{A}} \pi(a \mid s) = 1.$$

Definition 3.3. *The state-value function of a state s under a policy π, denoted by $v_\pi(s)$, is the expected return when starting in s and following π thereafter:*

$$v_\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]. \tag{3.6}$$

Here $\mathbb{E}_\pi$ denotes expectation with respect to the probability distribution induced by the policy π and the environment dynamics.

In addition to evaluating states, it is often useful to evaluate the quality of individual actions taken in a given state. This leads to the definition of the action-value function.

Definition 3.4. *The action-value function of an action a, given a state s under a policy π, denoted by $q_\pi(s, a)$, is the expected return when starting in s, taking action a and following π thereafter:*

$$q_\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]. \tag{3.7}$$

Using the recursive structure of the return specified in (3.5), both the state-value and action-value function can be calculated recursively:

$$\begin{aligned}
v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \qquad \text{(by (3.5))} \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s] \\
&= \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \mathbb{E}_\pi[G_{t+1} \mid S_{t+1} = s']\right] \\
&= \sum_a \pi(a \mid s) \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_\pi(s')\right].
\end{aligned} \tag{3.8}$$

Similarly, for the action-value function:

$$\begin{aligned}
q_\pi(s, a) &= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s, A_t = a] \qquad \text{(by (3.5))} \\
&= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\
&= \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \sum_{a'} \pi(a' \mid s') q_\pi(s', a')\right].
\end{aligned} \tag{3.9}$$

These equations are known as the Bellman equations and form the foundation of several reinforcement learning algorithms such as dynamic programming and temporal-difference learning [2, 5].

Lastly, it is often useful to consider the relative quality of an action compared to the policy's average behavior in that state. This is captured by the advantage function [9]:

$$A_\pi(s, a) = q_\pi(s, a) - v_\pi(s). \tag{3.10}$$

3.5 Value-Based Reinforcement Learning Algorithms

Value-based reinforcement learning methods aim to estimate the value functions introduced in section 3.4. A policy is then derived implicitly by selecting actions that maximize these estimated values.

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated
Initialize:
  $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
  $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):
  Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
      Append G to $Returns(S_t)$
      $V(S_t) \leftarrow$ average($Returns(S_t)$)

Figure 3.2: First-visit Monte Carlo prediction [25].

As explained in section 3.3, the objective of reinforcement learning is to maximize not only the immediate reward but the cumulative reward obtained over time. Consequently, an agent must balance selecting actions that appear to yield the highest return based on current knowledge (exploitation), also called the greedy action, with selecting alternative actions that may lead to better outcomes in the future (exploration).

A simple approach to enforcing exploration is the ϵ-greedy decision rule:

$$\pi(a \mid s) = \begin{cases} 1 - \varepsilon + \dfrac{\varepsilon}{|\mathcal{A}|}, & a = \arg\max_{a'} q(s, a'), \\ \dfrac{\varepsilon}{|\mathcal{A}|}, & \text{otherwise.} \end{cases}$$

More intuitively, this means that with probability $1 - \epsilon + \dfrac{\epsilon}{|\mathcal{A}|}$ the greedy action is chosen and any non-greedy choice has probability $\dfrac{\epsilon}{|\mathcal{A}|}$, with ties being broken randomly.

Two important classes of value-based methods are Monte Carlo methods and temporal-difference methods. Both aim to estimate value functions from experience, but differ in how these estimates are formed.

3.5.1 Monte Carlo Methods

Monte Carlo methods are a class of model-free reinforcement learning methods that learn from sampled episodes. They can be used both for policy evaluation (prediction) and for finding an optimal policy (control). In this thesis, only the prediction setting is considered, in which Monte Carlo methods are used to estimate value functions under a fixed policy.

The central idea of Monte Carlo prediction is to approximate value functions by averaging the returns observed in sampled trajectories. Given sufficiently many such trajectories, these estimates converge to the true value function by the law of large numbers.

A key characteristic of Monte Carlo methods is that updates are performed only after a complete episode has been observed, since the return G_t depends on future rewards.

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(terminal) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

Figure 3.3: TD(0) prediction [25].

Because the value estimates are obtained by averaging sampled returns, Monte Carlo methods provide unbiased estimates of the value function [24]. However, since updates require complete episodes, learning can be inefficient in tasks with long or continuing trajectories. Additionally, Monte Carlo methods are often noisy and have high variance. Temporal-difference learning addresses these limitations by updating value estimates after each step using bootstrapped targets.

3.5.2 Temporal-Difference Learning

Similarly to Monte Carlo methods, temporal-difference (TD) learning estimates value functions based on sampled experience. Monte Carlo methods approximate the value function v_π using the sampled return G_t, which requires waiting until the end of an episode. TD learning instead exploits the recursive structure of the Bellman equation and approximates

$$\mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s].$$

Since $v_\pi(S_{t+1})$ is unknown, TD learning replaces it with the current estimate $V(S_{t+1})$ and updates the value estimate using the quantity $R_{t+1} + \gamma V(S_{t+1})$. This allows value estimates to be updated after each transition, without waiting for the end of the episode, at the cost of introducing bias.

The term

$$\delta_t = R_{t+1} + \gamma V(S') - V(S_t)$$

in the update rule acts as an error and is called the TD error. It becomes relevant later for approximating the advantage function in policy gradient algorithms. Here S' denotes the successor state of S, which corresponds to S_{t+1} in the earlier notation.

One method that forms a bridge between Monte Carlo methods and TD(0) learning is called n-step TD learning. Whereas Monte Carlo methods use the full sampled return and TD(0) uses a one-step bootstrapped estimate, n-step TD methods use rewards observed several steps into the future.

Let

$$G_{t:t+1} = R_{t+1} + \gamma V(S_{t+1})$$

denote the one-step return used in TD(0). The subscript $t : t + 1$ denotes that it is the truncated reward starting from time t up until time $t + 1$, where $\gamma V(S_{t+1})$ approximates the remaining terms of the full return.

This can be extended to the n-step return:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \ldots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n}) \tag{3.11}$$

for all $n \geqslant 1$, $0 \leqslant t < T - n$.

Since these values are largely unknown at time t and only become available at time $t + n$, the n-step TD update rule is given by

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha\big[G_{t:t+n} - V_{t+n-1}(S_t)\big].$$

Lastly, it is also possible to average several n-step updates, which is called TD(λ), where $\lambda \in [0, 1]$:

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_{t:t+n} \tag{3.12}$$

where $G_{t:t+n} = G_t$ if $t+n \geqslant T$. This formulation closely resembles the generalized advantage estimator used in policy gradient methods such as Proximal Policy Optimization (PPO), which will be introduced later.

3.6 Policy-Based Reinforcement Learning Algorithms

Policy-based methods in general do not consult a value function to make decisions and instead directly learn a parameterized policy that provides probabilities for each action. It should be noted, however, that policy-based methods can still use value functions to learn the policy parameters, but they are not used directly to determine which actions to take. Such methods are called actor-critic methods, where the parameterized policy is the actor and the value function is the critic. The critic's role is to assess the quality of actions or states by estimating the expected return, thereby providing feedback that guides the actor's policy updates.

Let $\theta \in \mathbb{R}^d$ be the policy's parameter vector. Then

$$\pi(a \mid s, \theta) = \mathbb{P}(A_t = a \mid S_t = s, \theta_t = \theta)$$

is the probability that action a is taken when the environment is in state s at time t with parameter θ. Throughout this chapter, $\pi(a \mid s, \theta)$ is assumed to be differentiable with respect to θ, meaning that $\nabla\pi(a \mid s, \theta)$ exists and is finite for all $s \in \mathcal{S}$, $a \in \mathcal{A}$ and $\theta \in \mathbb{R}^d$.

A common class of policy-based methods are policy gradient methods, which optimize the policy parameters by following the gradient of a performance objective $J(\theta)$. This leads to update rules that approximate gradient ascent in J:

$$\theta_{k+1} = \theta_k + \alpha\widehat{\nabla J(\theta_k)}. \tag{3.13}$$

Here $\widehat{\nabla J(\theta)} \in \mathbb{R}^d$ is a stochastic estimate whose expectation approximates the gradient of $J(\theta)$ with respect to θ [25]. The definition of the performance measure depends on the application. Since this thesis only considers episodic tasks, the performance measure can be defined as $J(\theta) = v_{\pi_\theta}(s_0)$, where v_{π_θ} is the true value function of π_θ, the policy determined by θ. To perform gradient ascent as in (3.13), an expression for the gradient of the performance measure is required.

The policy gradient theorem provides a useful reformulation of this gradient that avoids differentiating the state distribution and instead expresses the gradient in terms of the policy and the action-value function [26].

Again following the notation from *Reinforcement Learning: An Introduction* [25], let $\eta(s)$ denote the average number of time steps spent in state s during an episode. Then

$$\mu(s) = \frac{\eta(s)}{\sum_{s'} \eta(s')}$$

for all $s \in \mathcal{S}$ is the state distribution, which represents the fraction of time spent in state s.

Using this notation, the policy gradient theorem states that

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a \mid s, \theta). \tag{3.14}$$

Since this expression represents a weighted sum over states according to their distribution, it can equivalently be written as an expectation over states encountered under policy π:

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s, a) \nabla \pi(a \mid s, \theta) = \mathbb{E}_\pi \left[\sum_a q_\pi(S_t, a) \nabla \pi(a \mid S_t, \theta) \right]. \tag{3.15}$$

3.6.1 REINFORCE

Multiplying by $1 = \frac{\pi(a|S_t,\theta)}{\pi(a|S_t,\theta)}$ inside the expectation in (3.15) introduces the policy probability as a weighting factor, allowing the sum over actions to be interpreted as an expectation over sampled actions:

$$\begin{aligned}
\nabla J(\theta) &\propto \mathbb{E}_\pi \left[\sum_a \pi(a \mid S_t, \theta)\, q_\pi(S_t, a)\, \frac{\nabla \pi(a \mid S_t, \theta)}{\pi(a \mid S_t, \theta)} \right] \\
&= \mathbb{E}_\pi \left[q_\pi(S_t, A_t)\, \frac{\nabla \pi(A_t \mid S_t, \theta)}{\pi(A_t \mid S_t, \theta)} \right] \qquad \text{(replacing } a \text{ by the sample } A_t \sim \pi) \\
&= \mathbb{E}_\pi \left[G_t\, \frac{\nabla \pi(A_t \mid S_t, \theta)}{\pi(A_t \mid S_t, \theta)} \right], \qquad \text{since } \mathbb{E}_\pi[G_t \mid S_t, A_t] = q_\pi(S_t, A_t).
\end{aligned} \tag{3.16}$$

This can now be plugged into (3.13), giving the REINFORCE update rule [28]:

$$\theta_{k+1} = \theta_k + \alpha G_t \frac{\nabla \pi(A_t \mid S_t, \theta)}{\pi(A_t \mid S_t, \theta)}. \tag{3.17}$$

Since the update uses the complete return G_t, REINFORCE is considered a Monte Carlo algorithm. Using the identity

$$\nabla \ln(x) = \frac{\nabla x}{x}$$

on the term $\frac{\nabla \pi(A_t \mid S_t, \theta)}{\pi(A_t \mid S_t, \theta)}$ gives the full REINFORCE algorithm for finding an optimal policy.

Lastly, it is also possible to introduce a baseline term into the policy gradient theorem, which is a common variance-reduction technique:

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a \big(q_\pi(s, a) - b(s)\big) \nabla \pi(a \mid s, \theta). \tag{3.18}$$

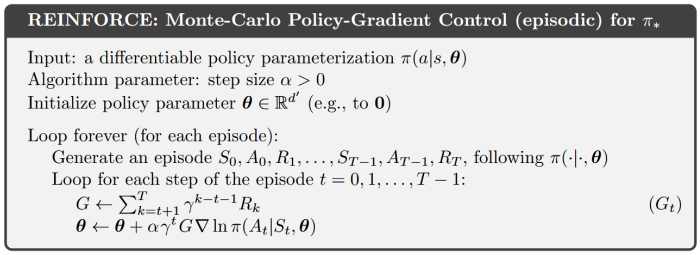


Figure 3.4: The REINFORCE algorithm for policy control [25].

This does not change the expected value of the policy gradient estimate, provided that the baseline is independent of the action a, since

$$\sum_a b(s)\,\nabla\pi(a \mid s,\theta) = b(s)\sum_a \nabla\pi(a \mid s,\theta) = b(s)\,\nabla\sum_a \pi(a \mid s,\theta) = b(s)\,\nabla 1 = 0.$$

Equivalently, the baseline term has expectation zero under the policy and therefore does not affect the expected update.

In practice, the baseline is often chosen as an estimate of the state-value function $v_\pi(s)$.

3.6.2 Generalized Advantage Estimation

The baseline can equivalently be introduced earlier in (3.16):

$$\begin{aligned}
\nabla J(\theta) &\propto \mathbb{E}_\pi\left[\sum_a \pi(a \mid S_t,\theta)\, q_\pi(S_t,a)\,\frac{\nabla\pi(a \mid S_t,\theta)}{\pi(a \mid S_t,\theta)}\right] \\
&= \mathbb{E}_\pi\left[q_\pi(S_t,A_t)\,\frac{\nabla\pi(A_t \mid S_t,\theta)}{\pi(A_t \mid S_t,\theta)}\right] \qquad \text{(replacing } a \text{ by the sample } A_t \sim \pi) \\
&= \mathbb{E}_\pi\left[\big(q_\pi(S_t,A_t) - b(S_t)\big)\,\frac{\nabla\pi(A_t \mid S_t,\theta)}{\pi(A_t \mid S_t,\theta)}\right]. \qquad (3.19)
\end{aligned}$$

Using the same identity $\nabla \ln(x) = \dfrac{\nabla x}{x}$ as earlier and choosing the baseline as the state-value function $v_\pi(s)$ makes it possible to express the gradient estimator in a more commonly used form by utilizing the advantage function introduced in section 3.4:

$$\nabla J(\theta) \propto \mathbb{E}_\pi\big[\nabla \ln \pi(A_t \mid S_t,\theta)\, A_\pi(S_t,A_t)\big]. \qquad (3.20)$$

Since the advantage function is generally unknown, it is in practice substituted by an estimate $\hat{A}_t$. A common approach to calculating this estimate is generalized advantage estimation, which reduces variance compared to other methods while keeping the bias sufficiently small [20].

For this, the TD error introduced in section 3.5.2,

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t),$$

is used. In particular, consider taking the sum of k of these δ terms, denoted by

$\hat{A}_t^{(k)}$:

$$\begin{aligned}
\hat{A}_t^{(1)} &= \delta_t = -V(S_t) + R_{t+1} + \gamma V(S_{t+1}),\\
\hat{A}_t^{(2)} &= \delta_t + \gamma\delta_{t+1} = -V(S_t) + R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}),\\
\hat{A}_t^{(3)} &= \delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2} = -V(S_t) + R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 V(S_{t+3}),\\
\hat{A}_t^{(k)} &= \sum_{l=0}^{k-1} \gamma^l \delta_{t+l} = -V(S_t) + R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{k-1} R_{t+k} + \gamma^k V(S_{t+k}),
\end{aligned}$$

where the equations result from a telescoping sum.

As k increases, the influence of the bootstrapped term $\gamma^k V(S_{t+k})$ decreases, so the estimator approaches the full Monte Carlo return, thereby decreasing the bias.

Taking $k \to \infty$ results in

$$\hat{A}_t^{(\infty)} = \sum_{l=0}^{\infty} \gamma^l \delta_{t+l} = -V(S_t) + \sum_{l=0}^{\infty} \gamma^l R_{t+l+1}. \tag{3.21}$$

The generalized advantage estimator is now defined as the exponentially weighted average of these k-step estimators:

$$\begin{aligned}
\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} &:= (1-\lambda)\big(\hat{A}_t^{(1)} + \lambda\hat{A}_t^{(2)} + \lambda^2\hat{A}_t^{(3)} + \cdots\big)\\
&= (1-\lambda)\big(\delta_t + \lambda(\delta_t + \gamma\delta_{t+1}) + \lambda^2(\delta_t + \gamma\delta_{t+1} + \gamma^2\delta_{t+2}) + \cdots\big)\\
&= (1-\lambda)\big(\delta_t(1 + \lambda + \lambda^2 + \cdots) + \gamma\delta_{t+1}(\lambda + \lambda^2 + \lambda^3 + \cdots)\\
&\qquad\qquad + \gamma^2\delta_{t+2}(\lambda^2 + \lambda^3 + \lambda^4 + \cdots) + \cdots\big)\\
&= (1-\lambda)\left(\delta_t\Big(\frac{1}{1-\lambda}\Big) + \gamma\delta_{t+1}\Big(\frac{\lambda}{1-\lambda}\Big) + \gamma^2\delta_{t+2}\Big(\frac{\lambda^2}{1-\lambda}\Big) + \cdots\right)\\
&= \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}.
\end{aligned} \tag{3.22}$$

This is closely related to the construction of TD(λ) introduced in section 3.5.2. It is important to note that this estimator is sensitive to the choice of hyperparameters λ and γ. In summary, generalized advantage estimation provides a practical way of constructing low-variance advantage estimates from temporal-difference errors, which will be used to formulate the PPO algorithm.

3.6.3 Proximal Policy Optimization

Since REINFORCE uses Monte Carlo estimates, it often suffers from high-variance gradient estimates, which can lead to unstable, overly large policy updates. Such large updates are problematic because the sampled trajectories and corresponding policy gradient estimates are only informative for policies close to the one that generated them. If the policy changes too much in a single update, the gradient estimate may no longer provide a reliable improvement direction, which can lead to unstable training or performance collapse. Proximal Policy Optimization (PPO) addresses this by limiting the size of each policy update, making training more stable and reliable [21].

Algorithm 1 PPO, Actor-Critic Style

for iteration=$1, 2, \ldots$ **do**
  for actor=$1, 2, \ldots, N$ **do**
    Run policy $\pi_{\theta_{\text{old}}}$ in environment for T timesteps
    Compute advantage estimates $\hat{A}_1, \ldots, \hat{A}_T$
  end for
  Optimize surrogate L wrt θ, with K epochs and minibatch size $M \leq NT$
  $\theta_{\text{old}} \leftarrow \theta$
end for

Figure 3.5: Proximal Policy Optimization for finding an optimal policy [21].

Let

$$r_t(\theta) = \frac{\pi(a_t \mid s_t, \theta)}{\pi(a_t \mid s_t, \theta_{\text{old}})}$$

denote the probability ratio, where θ_{old} is the policy parameter used to collect the current batch of trajectories. Thus, $r_t(\theta)$ measures how the probability assigned to action a_t in state s_t changes under the updated policy relative to the policy from which the data was sampled.

The goal of PPO is now to maximize the objective function

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t \right) \right]. \tag{3.23}$$

Here $\hat{\mathbb{E}}_t$ is the empirical average over a finite batch of samples. Furthermore, $\hat{A}_t$ is the advantage function, which is most commonly estimated via generalized advantage estimation as introduced in section 3.6.2 and ϵ is a hyperparameter, for example $\epsilon = 0.2$. The clipping function is defined as

$$\text{clip}(x, a, b) = \begin{cases} a, & x < a, \\ x, & a \leqslant x \leqslant b, \\ b, & x > b. \end{cases}$$

Intuitively, the first part of this objective function, $r_t(\theta)\hat{A}_t$, encourages changes to the policy that increase the probability of advantageous actions and decrease the probability of disadvantageous ones relative to the old policy. Without further constraints, this term can favor increasingly large policy updates whenever they appear to improve the objective. Therefore, the second term,

$$\text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t,$$

truncates the probability ratio and removes the incentive to move $r_t(\theta)$ outside the interval $[1-\epsilon, 1+\epsilon]$. Taking the minimum of the clipped and unclipped objectives acts as a lower, or pessimistic, bound on the update and prevents excessively large updates, making PPO more stable.

In summary, this chapter introduced the main reinforcement learning concepts needed for the rest of the thesis. The reinforcement learning problem was formalized as a Markov decision process and both value-based and policy-based methods were discussed. In particular, temporal-difference learning and generalized advantage estimation were introduced as important building blocks for actor-critic methods, while Proximal Policy Optimization was presented as a policy-based algorithm that stabilizes training by constraining policy updates. In the following chapters, the portfolio optimization problem will

be formulated as a reinforcement learning task and the methods introduced in this chapter will be used to compare a reinforcement learning approach with the classical Markowitz framework.

Chapter 4

Experimental Setup

4.1 Introduction

This chapter describes the experimental setup used to compare the reinforcement learning approach with the classical Markowitz approach. First, the data used in the experiments and the selected asset universe are introduced. The chapter then explains how the two portfolio construction methods are implemented in practice. Finally, the evaluation procedure is presented.

4.2 Data and Asset Universe

The empirical analysis is based on daily adjusted closing prices obtained via Yahoo Finance. From these prices, simple daily returns are computed and used throughout the experiments.

The first part of the empirical analysis considers data from 2011 to April 2026. In this setting, the reinforcement learning agent was trained up to the end of 2018 and both the Markowitz and reinforcement learning portfolios were then evaluated from 2019 onwards. A second experiment considers an earlier market regime. Here, the training period extends from 1998 to 2006, while the test period covers 2007 to the beginning of 2015, thereby including the global financial crisis in 2008.

The asset universe in both cases consists of large-cap U.S. equities chosen to include different types of firms, such as long-term growth stocks like Microsoft or Apple, healthcare companies such as Johnson & Johnson, financial firms like JPMorganChase, energy stocks like Exxon Mobil and firms with prolonged weak performances like Ford in the 2011-2026 universe or General Electric in the 1998-2015 universe. This was done to avoid a universe dominated exclusively by highly successful technology firms and to obtain a more informative comparison between the considered portfolio strategies. The complete asset lists used in the different experimental settings are reported in Appendix A.

In addition, the experiments are conducted for two different decision frequencies, namely daily and monthly rebalancing. Moreover, each approach is evaluated both with and without transaction costs, since the impact of such costs may differ across strategies.

4.3 General Experimental Framework

Despite the differences between the Markowitz and reinforcement learning approaches, both are evaluated under shared modeling assumptions. In particular, both methods are based on the same daily asset return series, the same asset universes and the same transaction-cost specification.

Let $R_t = (R_{1,t}, R_{2,t}, \dots, R_{d,t})$, $t = 1, \dots, T$ denote the vector of daily asset returns where d corresponds to the number of assets. Furthermore, let $\pi_t = (\pi_{1,t}, \pi_{2,t}, \dots, \pi_{d,t})$ be the current portfolio weights at time t as introduced in chapter 2, before the realization of the asset returns R_t. The realized portfolio return at time t is then given by $R_t^{\pi} = \pi_t' R_t$.

A portfolio strategy does not necessarily update its allocation on every trading day. Depending on the experimental setting, rebalancing decisions are made either daily or on the last trading day of each month. In the monthly setting, the portfolio weights are determined using information available up to the end of the rebalancing day. The newly chosen weights are then applied from the following trading day onward.

Between rebalancing dates, the portfolio is not actively traded. However, its weights still change mechanically because assets earn different returns. For example, an asset with a high realized return will represent a larger share of the portfolio value after the holding period. This passive change in portfolio weights is referred to as portfolio drift.

To distinguish between the portfolio before and after rebalancing, let $\tilde{\pi}_t$ denote the portfolio weights immediately before trading at a rebalancing date t. These are the drifted weights implied by the previously held portfolio and the asset returns realized since the last rebalance.

Given portfolio weights π_s on day s, the one-day drift update is given by

$$\pi_{i,s}^{*} = \frac{\pi_{i,s}(1 + R_{i,s})}{\sum_{j=1}^{d} \pi_{j,s}(1 + R_{j,s})}. \tag{4.1}$$

In the daily rebalancing case, the pre-trade weights at the next rebalancing date satisfy $\tilde{\pi}_t = \pi_{t-1}^{*}$. In the monthly case, repeated application of this update over the holding period yields the pre-trade weights $\tilde{\pi}_t$.

The portfolio turnover at a rebalancing date t is then defined as

$$\text{Turnover}_t = \frac{1}{2} \sum_{i=1}^{d} |\pi_{i,t} - \tilde{\pi}_{i,t}|. \tag{4.2}$$

Thus, turnover measures the amount of trading required to move from the passively drifted portfolio to the new target portfolio [19].

The transaction cost incurred at time t is modeled as a proportional turnover cost

$$\text{Cost}_t = c \cdot \text{Turnover}_t, \tag{4.3}$$

where $c = 0.001$ denotes the transaction cost parameter used throughout all experiments. This corresponds to transaction costs of 1 dollar per 1,000 dollars traded.

In the implementation, this cost is deducted from the first realized portfolio return after rebalancing, which is done consistently throughout all experiments that include transaction costs.

4.4 Markowitz Setup

The general framework described above must now be specified for the Markowitz approach. In particular, this requires a practical choice of how the expected returns and covariance matrix are estimated over time.

As explained in section 2.3, the Markowitz model requires estimates of the expected returns and covariances of the individual assets. In the implementation, these quantities are obtained using a rolling-window approach. More precisely, for each rebalancing date, a trailing estimation window of length L, which was set to either 60 or 252 trading days, is considered. The estimation window therefore consists of the return vectors

$$(R_{t-L+1}, R_{t-L+2}, \dots, R_t) \tag{4.4}$$

so that parameter estimation is based on all information available up to and including the rebalancing day. Within this window, the expected return vector is estimated by the historical sample mean, while the covariance matrix is estimated using the Ledoit–Wolf shrinkage estimator. In the monthly rebalancing case, the estimated expected return vector and covariance matrix are scaled according to the length of the corresponding holding period. The rolling-window approach was chosen in order to base parameter estimation on more recent market information, rather than on the full return history. The specific window length was selected based on preliminary experiments.

In the transaction-cost-adjusted specification, the Markowitz objective is extended by a turnover-based cost term. The optimization problem therefore becomes

$$\max_{\pi \in \mathbb{R}^d} \mu'\pi - \frac{1}{2}\lambda\pi'\sigma\pi - \text{Cost}. \tag{4.5}$$

For the comparison, λ is chosen such that the resulting Markowitz portfolios approximately match target volatility levels of 15%, 20% and 30%. Both the λ values and the estimation window length were selected based on their realized empirical behavior within the respective evaluation regime. The Markowitz portfolios should therefore be treated as competitive regime-specific benchmarks, rather than as the output of a fully out-of-sample procedure.

4.5 Reinforcement Learning Setup

Apart from the general framework described above, the reinforcement learning approach additionally requires an explicit model of the environment in which the agent interacts. As in the Markowitz implementation, the state representation is based on a trailing window of historical returns. In all experiments, the corresponding window length is set to $L = 60$. However, in contrast to the Markowitz approach, the reinforcement learning agent does not summarize this information through estimated moments. Instead, it directly observes the return window itself, together with the portfolio weights currently held.

No additional market features or model-based quantities are included in the state representation. This restriction is imposed deliberately in order to examine whether the agent can learn useful allocation rules directly from historical return information and the current portfolio composition alone.

The action of the agent is an unconstrained real-valued vector. In order to obtain admissible portfolio weights, the softmax function, which is defined as

$$\sigma_i(z) = \frac{e^{z_i}}{\sum_{j=1}^{d} e^{z_j}} \tag{4.6}$$

is applied to the raw action output. This guarantees that the resulting portfolio is long-only and fully invested.

The reward is defined in terms of logarithmic portfolio returns. In the daily rebalancing case, the reward at time t is given by $\log(1 + R_t^\pi)$. In the monthly rebalancing case, the reward is given by the sum of daily log returns over the holding period between two consecutive rebalancing dates. Logarithmic returns were chosen as they naturally penalize volatility [17]. If transaction costs are included, the corresponding turnover-based cost term is deducted from the first realized portfolio return after rebalancing. Since this term is used in the reward function, the agent is incentivized to avoid unnecessarily aggressive rebalancing.

Training is carried out using Proximal Policy Optimization (PPO) as implemented in Stable-Baselines3. During training, episodes have a length of 252 trading days. Their starting points are sampled randomly from the training period, subject to the chosen rebalancing frequency. The specific hyperparameters used for training are reported in Appendix A. After training, the learned policy is evaluated on the test period without further updating.

Since reinforcement learning training involves random initialization and stochastic optimization, results are reported for three different models, corresponding to three random seeds. These were chosen to represent one above-average run, one approximately average run and one below-average run, where 10 runs were considered for each experiment.

4.6 Evaluation

The resulting portfolios of both strategies are evaluated on the test period through the metrics introduced in section 2.4. For the Sharpe ratio, the risk-free rate is proxied by the yield on the 13-week U.S. Treasury bill, obtained via the ticker ^IRX via Yahoo Finance. Since the available series contains occasional missing observations, a single representative risk-free rate is used in the empirical evaluation, namely the average of the corresponding daily risk-free rates over the respective test period. Whenever applicable, the annualized turnover is also reported.

In addition, an equally weighted benchmark portfolio is considered, which acts as a simple and stable comparison. This benchmark is rebalanced at the same frequency as the portfolios used and is subject to the same transaction cost-convention, where transaction costs occur due to the previously explained portfolio drift.

With the experimental setup now fully specified, the empirical comparison can be carried out. The next chapter presents the resulting portfolio performance of the Markowitz, reinforcement learning and equally weighted benchmark strategies across the considered experimental settings.

Chapter 5

Results

This chapter presents the empirical results of the portfolio comparison. The chapter is divided into two parts corresponding to the two market regimes introduced in the previous chapter. For each regime, four experimental settings are considered: daily and monthly rebalancing, each evaluated with and without transaction costs.

5.1 Results for the 2019 - 2026 Market Regime

The first experiment considers the 2019-2026 market regime under daily rebalancing without transaction costs. The cumulative wealth paths of the reinforcement learning, Markowitz and equally weighted portfolios are shown in Figure 5.1, followed by the corresponding performance metrics in Table B.4. All cumulative wealth plots are normalized to an initial portfolio value of 1,000.

5.1.1 Daily Rebalancing without Transaction Costs

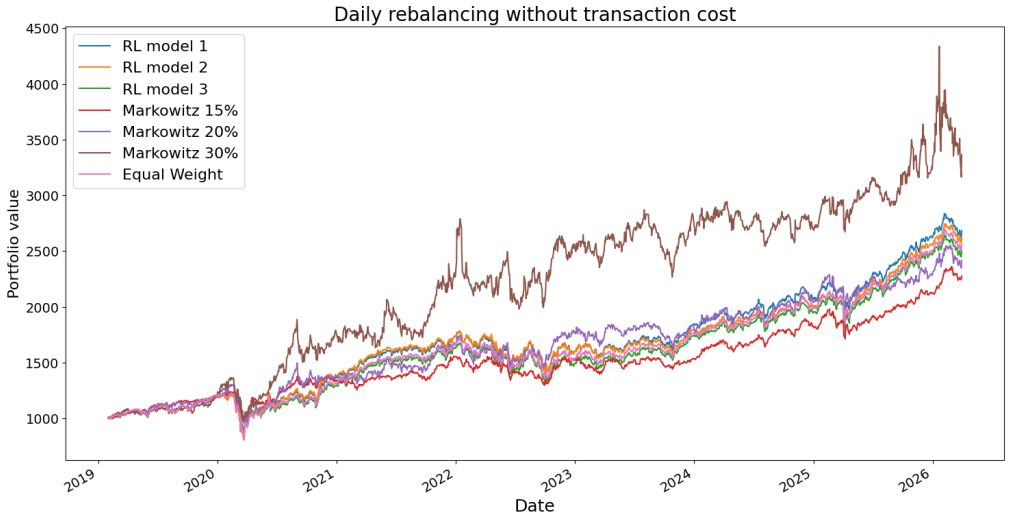


Figure 5.1: Daily rebalancing without transaction costs

The results show that the reinforcement learning portfolios are competitive in this setting, particularly when evaluated by the Sharpe ratio. Two of the

Table 5.1: Daily rebalancing without transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD
RL Seed 1	1.685	0.148	0.186	0.697	0.337
RL Seed 2	1.624	0.145	0.186	0.678	0.337
RL Seed 3	1.502	0.137	0.181	0.655	0.305
Markowitz 15%	1.277	0.122	0.150	0.669	0.225
Markowitz 20%	1.401	0.130	0.200	0.583	0.268
Markowitz 30%	2.367	0.185	0.301	0.630	0.293
Equal Weight	1.560	0.141	0.183	0.668	0.337

three RL runs achieve the highest Sharpe ratios among all reported strategies, while the third run remains slightly below the Markowitz 15% portfolio and the equally weighted benchmark. On average across all 10 runs performed, RL achieved a Sharpe ratio of 0.675, beating out Markowitz and equal weight on a risk adjusted basis. In terms of raw performance, however, the Markowitz portfolio with a 30% target volatility achieves the highest cumulative return and CAGR, reflecting the larger risk exposure induced by the higher volatility target.

At the same time, the reinforcement learning portfolios exhibit larger maximum drawdowns than the Markowitz portfolios. This suggests that the RL strategies are able to generate attractive average compensation per unit of volatility but do not necessarily control downside risk as effectively as the Markowitz benchmark. The result therefore highlights a trade-off between risk-adjusted performance measured by the Sharpe ratio and downside risk measured by maximum drawdown.

5.1.2 Daily Rebalancing with Transaction Costs

Table 5.2: Daily rebalancing with transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD	Turnover
RL Seed 1	1.312	0.124	0.183	0.590	0.337	16.809
RL Seed 2	1.270	0.122	0.183	0.577	0.337	15.458
RL Seed 3	1.206	0.117	0.184	0.553	0.348	17.888
Markowitz 15%	1.184	0.116	0.150	0.630	0.225	2.519
Markowitz 20%	1.420	0.132	0.200	0.587	0.287	4.459
Markowitz 30%	1.516	0.138	0.301	0.493	0.338	5.156
Equal Weight	1.537	0.139	0.183	0.661	0.337	1.262

After including transaction costs, the relative ranking changes substantially. The equally weighted benchmark achieves the highest Sharpe ratio, while the reinforcement learning portfolios lose much of their advantage from the cost-free setting. This is mainly explained by the substantially higher turnover of

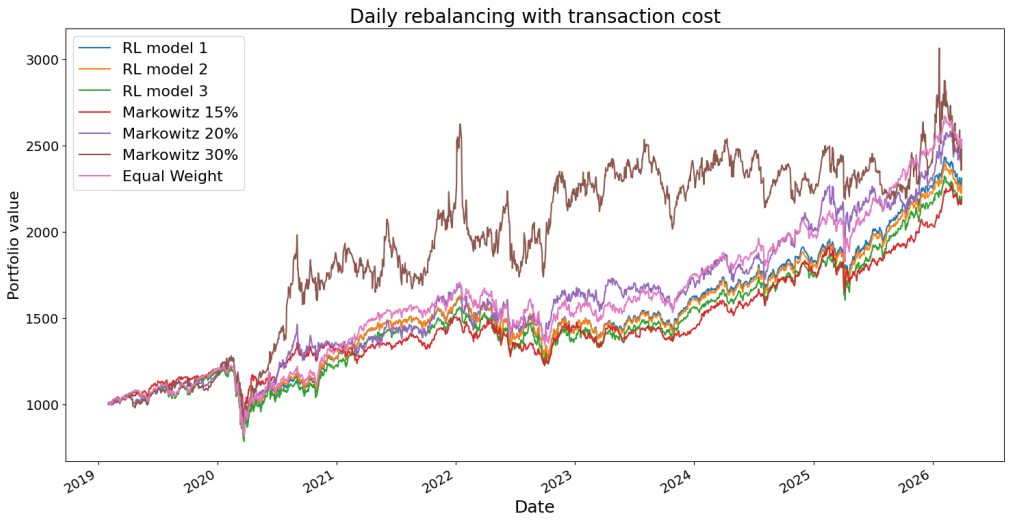


Figure 5.2: Daily rebalancing with transaction costs

the RL strategies. The annualized turnover of the RL portfolios lies between 15 and 18, meaning that the strategies trade roughly 15 to 18 times the portfolio value per year.

The result suggests that the daily reinforcement learning strategies rely heavily on frequent reallocations. While such flexibility can be beneficial in the absence of costs, it becomes less attractive once trading costs are taken into account. The equal-weight benchmark, by contrast, benefits from its comparatively low turnover and therefore remains competitive despite its simple construction.

A similar disadvantage can be seen in the 20 and 30% volatility Markowitz portfolios, leading to even the 30% volatility target underperforming in terms of cumulative return and CAGR when compared to the equal weight portfolio. The 15% variant, however, produces the second best results overall on Sharpe ratio and still manages downside risk in terms of maximum drawdown considerably better than its counterparts.

5.1.3 Monthly Rebalancing without Transaction Costs

For monthly rebalancing, the Markowitz portfolio is evaluated using two different estimation windows, namely 60 and 252 trading days. The 60-day window represents a more reactive specification, as it gives greater weight to recent market developments. However, it also leads to noisier estimates of expected returns and covariances. In contrast, the 252-day window corresponds approximately to one trading year and provides more stable parameter estimates but may react more slowly to changes in market conditions. In several experiments, the 60-day window produces stronger performance than the 252-day window. This should not be interpreted as general evidence that shorter estimation windows are superior. Rather, it suggests that the more reactive specification was better suited to this particular market regime.

Here, Markowitz 1 corresponds to a lookback window of 252 days, whereas Markowitz 2 corresponds to a lookback window of 60 days. In this particular

Table 5.3: Markowitz estimation window comparison without transaction cost

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD
Markowitz 1 15%	1.224	0.118	0.150	0.647	0.233
Markowitz 1 20%	1.500	0.137	0.200	0.611	0.317
Markowitz 1 30%	1.900	0.161	0.300	0.559	0.349
Markowitz 2 15%	1.230	0.119	0.150	0.649	0.230
Markowitz 2 20%	1.752	0.152	0.200	0.678	0.264
Markowitz 2 30%	2.811	0.206	0.300	0.688	0.342

variation the 60 day estimation window outperforms across all metrics. Since the purpose of the Markowitz benchmark is to provide a competitive classical reference, rather than a directly investible trading rule, the better-performing estimation window specification within each regime is used in the main comparison. This choice should not be interpreted as evidence that an investor could have selected the optimal window in real time.

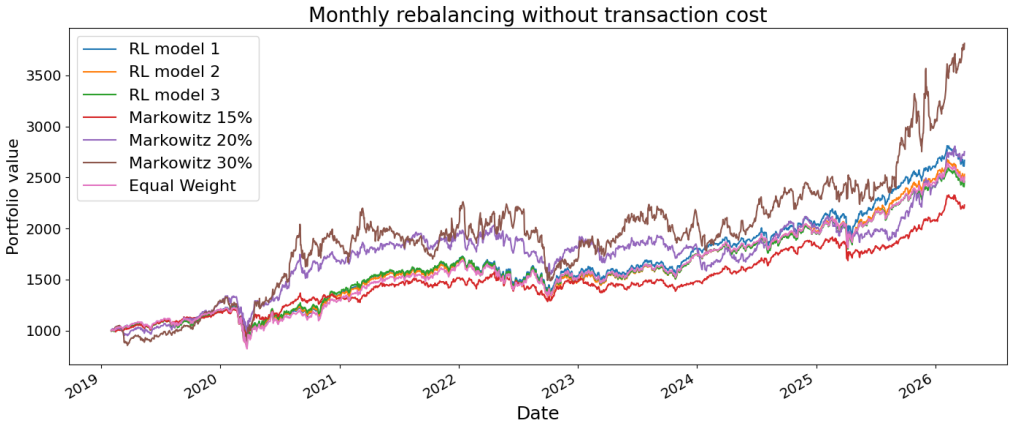


Figure 5.3: Monthly rebalancing without transaction costs

Table 5.4: Monthly rebalancing without transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD
RL Seed 1	1.667	0.147	0.182	0.704	0.323
RL Seed 2	1.529	0.139	0.181	0.665	0.312
RL Seed 3	1.462	0.134	0.180	0.646	0.296
Markowitz 2 15%	1.230	0.119	0.150	0.649	0.230
Markowitz 2 20%	1.752	0.152	0.200	0.678	0.264
Markowitz 2 30%	2.811	0.206	0.300	0.688	0.342
Equal Weight	1.502	0.137	0.181	0.655	0.332

In the monthly rebalancing case without transaction costs both Markowitz and RL are competitive, with the best reinforcement learning run achieving the

highest overall Sharpe ratio. In this setting, the 30% Markowitz target achieved the second highest Sharpe ratio, while also having the highest maximum drawdown. Overall, however, with more conservative volatility levels chosen it still manages downside risk better than both the reinforcement learning agent and the equal weight benchmark.

5.1.4 Monthly Rebalancing with Transaction Costs

Table 5.5: Markowitz estimation window comparison with transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD	Turnover
Markowitz 1 15%	1.190	0.116	0.150	0.633	0.233	1.507
Markowitz 1 20%	1.438	0.133	0.200	0.594	0.311	3.030
Markowitz 1 30%	1.798	0.155	0.300	0.543	0.355	4.220
Markowitz 2 15%	1.174	0.115	0.150	0.626	0.231	3.915
Markowitz 2 20%	1.665	0.147	0.200	0.656	0.264	6.440
Markowitz 2 30%	2.589	0.196	0.300	0.660	0.346	8.492

Including transaction costs changes the comparison between the two estimation windows. For the 15% target volatility, the 252-day window achieves a slightly higher Sharpe ratio and only a marginally larger maximum drawdown than the 60-day window, while also producing substantially lower turnover. For the 20% and 30% target volatility levels, however, the 60-day window continues to deliver higher cumulative returns, higher CAGR and higher Sharpe ratios, despite its higher turnover. This suggests that the more reactive 60-day specification remains advantageous at higher target volatility levels in this regime but that this benefit comes at the cost of increased trading activity.

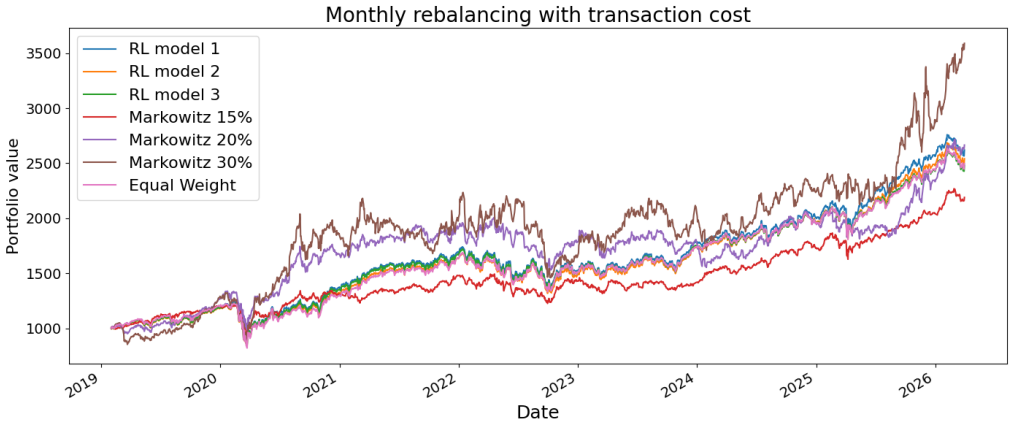


Figure 5.4: Monthly rebalancing with transaction costs

In the last setting considered for this market regime, the reinforcement learning portfolios remain competitive. The strongest and average RL runs outperform both the Markowitz specifications and the equal-weight portfolio in terms of Sharpe ratio, while the weakest RL run remains close to the benchmark. This contrasts with the daily rebalancing case with transaction costs,

Table 5.6: Monthly rebalancing with transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD	Turnover
RL Seed 1	1.620	0.144	0.181	0.691	0.318	1.914
RL Seed 2	1.541	0.139	0.181	0.668	0.312	1.915
RL Seed 3	1.480	0.136	0.180	0.651	0.312	1.876
Markowitz 1 15%	1.190	0.116	0.150	0.633	0.233	1.507
Markowitz 2 20%	1.665	0.147	0.200	0.656	0.264	6.440
Markowitz 2 30%	2.589	0.196	0.300	0.660	0.346	8.492
Equal Weight	1.497	0.137	0.181	0.654	0.332	0.288

where the RL portfolios were much more strongly affected by their high trading activity.

The difference is mainly visible in turnover. Under monthly rebalancing, the annualized turnover of the RL strategies falls to roughly 1.9, compared with values above 15 in the daily transaction-cost setting. As a result, the transaction-cost burden is substantially lower, allowing the RL strategies to retain much of their performance. The Markowitz portfolios with 15% and 20% target volatility continue to exhibit lower maximum drawdowns than the RL portfolios, while the 20% and 30% Markowitz specifications outperform the equal-weight benchmark in terms of Sharpe ratio.

5.2 Results for the 1998-2015 Market Regime

The second part of this chapter considers a training period from 1998 to 2006 and a test period from 2007 to the beginning of 2015. This period includes the global financial crisis in 2008, leading to generally higher volatility and maximum drawdowns when compared to the 2019 - 2026 regime. Furthermore, direct comparisons between cumulative returns should be avoided, as the length of the two periods differs. Sharpe ratios are also interpreted within each regime, since both market conditions and risk-free rates differ across the two periods.

5.2.1 Monthly Rebalancing with Transaction Costs

To avoid repeating the full experimental grid, the main text focuses on the monthly rebalancing specification with transaction costs for the 2007-2015 regime. This specification is particularly informative because it includes trading frictions while avoiding the excessive turnover of daily rebalancing. The remaining variants for this regime are reported in Appendix B.

The 30% volatility Markowitz portfolio with a 252-day estimation window stands out in this regime. It achieves the highest cumulative return, CAGR and Sharpe ratio among the Markowitz specifications. This result appears to be driven by the combination of a large risk budget and a relatively stable one-year estimation window, which allowed the portfolio to participate strongly in

Table 5.7: Markowitz estimation window comparison with transaction costs 2007-2015

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD	Turnover
Markowitz 1 15%	1.038	0.094	0.150	0.624	0.329	1.440
Markowitz 1 20%	1.444	0.120	0.200	0.627	0.415	3.140
Markowitz 1 30%	3.211	0.199	0.300	0.731	0.534	4.022
Markowitz 2 15%	1.122	0.100	0.151	0.655	0.370	3.118
Markowitz 2 20%	1.456	0.120	0.200	0.629	0.452	6.619
Markowitz 2 30%	1.994	0.149	0.300	0.587	0.586	7.436

the post-crisis recovery while avoiding the higher turnover of the shorter 60-day window.

However, this result should be interpreted cautiously. The same portfolio also exhibits a maximum drawdown above 50%, indicating substantial downside exposure. The strong performance therefore does not represent a uniformly superior risk profile compared to other target volatility levels but rather a high-risk allocation that was rewarded in this particular market regime.

Furthermore, it should be noted that while the 60-day window outperforms on Sharpe ratio in the 15 and 20% case, the realized volatility could not be reduced exactly to the 15% target, as the inherent noise in the data led to too much volatility, even for very large values of the risk-aversion parameter. The higher maximum drawdowns of the 60-day specifications across all target volatility levels further indicate that the shorter window produces less stable allocations in this regime.

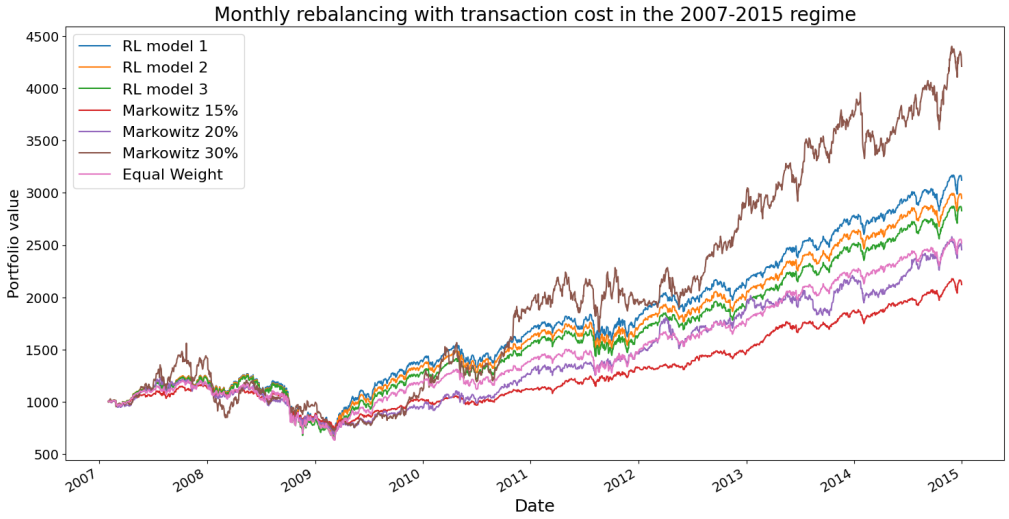


Figure 5.5: Monthly rebalancing with transaction costs in the 2007-2015 regime

In this regime, reinforcement learning is again able to compete with the Markowitz portfolio, its best run having the highest Sharpe ratio while also

Table 5.8: Monthly rebalancing with transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD	Turnover
RL Seed 1	2.121	0.155	0.218	0.733	0.453	1.724
RL Seed 2	1.947	0.146	0.218	0.701	0.465	1.688
RL Seed 3	1.829	0.140	0.219	0.675	0.490	1.761
Markowitz 2 15%	1.122	0.100	0.151	0.655	0.370	3.118
Markowitz 2 20%	1.456	0.120	0.200	0.629	0.452	6.619
Markowitz 1 30%	3.211	0.199	0.300	0.731	0.534	4.022
Equal Weight	1.519	0.124	0.209	0.627	0.477	0.254

managing downside risk better when compared to the 30% volatility portfolio, which produced the second highest Sharpe ratio. Across all methods, the maximum drawdown is significantly higher than in the 2019-2026 regime, reflecting the stress of the 2008 financial crisis within the test period. The equal-weight benchmark performs less favorably in this regime than in the 2019-2026 setting. It achieves the lowest Sharpe ratio among the reported strategies, although the difference relative to the 20% Markowitz portfolio is small.

The additional results reported in the appendix show that the strong performance of the high-volatility Markowitz specification in this regime is not limited to monthly rebalancing. In particular, the daily rebalancing results indicate that a more reactive Markowitz strategy with a larger risk budget was especially well suited to this historical period.

Chapter 6

Limitations

The empirical results should be interpreted in light of several limitations. First, the reinforcement learning results depend strongly on the chosen training procedure. PPO training is stochastic and sensitive to hyperparameters, network architecture, reward specification, training length and random initialization. The reported RL results therefore represent the performance of the trained PPO agents under the specified experimental setup, rather than a globally optimal reinforcement learning portfolio strategy.

Second, several model choices were selected based on preliminary empirical performance. This includes the Markowitz risk-aversion parameter, the estimation-window length and the PPO hyperparameters. While this makes it possible to compare reasonably competitive specifications, it also means that some choices may be specific to the considered market regimes and may not generalize to future unseen data.

Third, the asset universe is manually selected and consists of large U.S. equities with available historical data. This may introduce survivorship and selection bias, since the universe does not necessarily reflect the exact set of assets available to investors at each point in time. This limitation is particularly relevant for the 2007-2015 test period, as all assets considered survived the financial crisis and are still actively traded today. This may make aggressive allocations within the selected universe appear more favorable than they would have been in a historically complete investment universe. The results therefore should not be interpreted as representative of the entire market.

Fourth, the RL environment uses a deliberately simple state representation consisting only of historical returns and current portfolio weights. This may restrict the agent's ability to learn more complex market relationships. Similarly, the action is modeled as an unconstrained vector that is transformed into portfolio weights using the softmax function. While this enforces the long-only and fully invested constraints, it does not directly control turnover. As a result, especially in daily rebalancing settings, the learned policy generates frequent reallocations, leading to substantial turnover and, when transaction costs are included, a considerable cost drag.

Finally, the transaction cost modeling is simplified. While the proportional turnover cost used captures the basic effect of trading frictions, it does not account for the fact that actual trading costs may vary across assets and over time.

Chapter 7

Conclusion and Outlook

This thesis investigated the application of reinforcement learning to portfolio optimization under different rebalancing frequencies, market regimes and transaction-cost structures. The empirical analysis compared a Proximal Policy Optimization agent with Markowitz portfolios and an equally weighted benchmark across daily and monthly rebalancing settings.

The results suggest that reinforcement learning can be competitive in portfolio optimization, especially in the monthly rebalancing setting. Across the considered market regimes, the stronger RL runs often achieved Sharpe ratios comparable to or above those of the Markowitz specifications and the equal-weight benchmark. This indicates that the agent was able to extract useful allocation rules from historical return information and current portfolio weights. The Markowitz portfolios, by contrast, provided a useful benchmark with explicit control over volatility targets, but their performance depended strongly on the chosen risk-aversion parameter and estimation window.

The central finding of the thesis concerns the interaction between rebalancing frequency and transaction costs. In the daily rebalancing setting without transaction costs, the RL portfolios were competitive. Once transaction costs were included, however, their performance deteriorated considerably. This was mainly caused by high turnover, indicating that the learned daily policies relied on frequent reallocations. Therefore, the findings imply that transaction costs are a key limitation for the reinforcement-learning approach considered here. The chosen action representation, combined with the softmax transformation, did not sufficiently restrict turnover in the daily setting. In contrast, monthly rebalancing reduced turnover substantially and allowed the RL strategies to retain much of their performance after transaction costs. Thus, the practical usefulness of the approach depends not only on the agent's ability to generate profitable allocation signals but also on whether these signals can be implemented with realistic trading costs. Overall, the results show that reinforcement learning can be effective in this setting but only when trading activity is sufficiently controlled.

Future research could extend the analysis in several directions. One promising direction is to improve the RL action design, for example by modeling trades directly or imposing explicit turnover constraints. Another possibility is to enrich the state representation with additional market information, such as volatility estimates or macroeconomic variables. Finally, future work could evaluate the strategies on broader asset universes and additional market regimes to assess the robustness of the findings.

Bibliography

[1] Richard Bellman. 1957. A Markovian Decision Process. *Journal of Mathematics and Mechanics*, 6, 5, 679–684. ISSN: 00959057, 19435274. Retrieved 03/07/2026 from http://www.jstor.org/stable/24900506.

[2] Richard Bellman. 1957. *Dynamic Programming*. (1st ed.). Princeton University Press, Princeton, NJ, USA.

[3] Fischer Black and Robert Litterman. 1992. Global Portfolio Optimization. *Financial Analysts Journal*, 48, 5, (September 1992), 28–43. ISSN: 1938-3312. DOI: 10.2469/faj.v48.n5.28. http://dx.doi.org/10.2469/faj.v48.n5.28.

[4] Ricard Durall. 2022. Asset Allocation: From Markowitz to Deep Reinforcement Learning. (2022). arXiv: 2208.07158 [q-fin.PM]. https://arxiv.org/abs/2208.07158.

[5] Ronald A. Howard. 1960. *Dynamic Programming and Markov Processes*. Technology Press of the Massachusetts Institute of Technology, Cambridge, MA.

[6] Jinkyu Kim, Hyunjung Yi, Mogan Gim, Donghee Choi, and Jaewoo Kang. 2025. DeepAries: Adaptive Rebalancing Interval Selection for Enhanced Portfolio Selection. (2025). arXiv: 2510.14985 [q-fin.PM]. https://arxiv.org/abs/2510.14985.

[7] Ralf Korn. 2014. *Moderne Finanzmathematik – Theorie und praktische Anwendung: Band 1 – Optionsbewertung und Portfolio-Optimierung*. Springer Fachmedien Wiesbaden. ISBN: 9783658041274. DOI: 10.1007/978-3-658-04127-4. http://dx.doi.org/10.1007/978-3-658-04127-4.

[8] Olivier Ledoit and Michael Wolf. 2004. Honey, I Shrunk the Sample Covariance Matrix. *The Journal of Portfolio Management*, 30, 4, (July 2004), 110–119. ISSN: 2168-8656. DOI: 10.3905/jpm.2004.110. http://dx.doi.org/10.3905/jpm.2004.110.

[9] III Leemon C. Baird. 1993. Advantage Updating. Technical report WL-TR-93-1146. Wright Laboratory, Wright-Patterson Air Force Base, OH.

[10] Qing Lim, Qi Cao, and Chai Quek. 2022. Dynamic portfolio rebalancing through reinforcement learning. *Neural Computing and Applications*, 34, (May 2022). DOI: 10.1007/s00521-021-06853-3.

[11] John Lintner. 1965. The Valuation of Risk Assets and the Selection of Risky Investments in Stock Portfolios and Capital Budgets. *The Review of Economics and Statistics*, 47, 1, (February 1965), 13. ISSN: 0034-6535. DOI: 10.2307/1924119. http://dx.doi.org/10.2307/1924119.

[12] Harry Markowitz. 1952. Portfolio Selection. *The Journal of Finance*, 7, 1, (March 1952), 77. ISSN: 0022-1082. DOI: 10.2307/2975974. http://dx.doi.org/10.2307/2975974.

[13] Robert C. Merton. 1969. Lifetime Portfolio Selection under Uncertainty: The Continuous-Time Case. *The Review of Economics and Statistics*, 51, 3, (August 1969), 247. ISSN: 0034-6535. DOI: 10.2307/1926560. http://dx.doi.org/10.2307/1926560.

[14] Robert C. Merton. 1980. On estimating the expected return on the market: An exploratory investigation. *Journal of Financial Economics*, 8, 4, 323–361. ISSN: 0304-405X. DOI: https://doi.org/10.1016/0304-405X(80)90007-0. https://www.sciencedirect.com/science/article/pii/0304405X80900070.

[15] Richard Michaud. 1989. The Markowitz Optimization Enigma: Is 'Optimized' Optimal? *Financial Analysts Journal - FINANC ANAL J*, 45, (January 1989), 31–42. DOI: 10.2469/faj.v45.n1.31.

[16] Jan Mossin. 1966. Equilibrium in a Capital Asset Market. *Econometrica*, 34, 4, (October 1966), 768. ISSN: 0012-9682. DOI: 10.2307/1910098. http://dx.doi.org/10.2307/1910098.

[17] Ayobami Gabriel Olanrewaju, Adeyinka Oluwasimisola Ajayi, Omolabake Ibironke Pacheco, Adebayo Oluwatosin Dada, and Adepeju Ayotunde Adeyinka. 2025. AI-driven adaptive asset allocation: A machine learning approach to dynamic portfolio optimization in volatile financial markets. *International Journal of Research in Finance and Management*, 8, 1, 320–332. DOI: 10.33545/26175754.2025.v8.i1d.451.

[18] Martin L. Puterman. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Wiley, (April 1994). ISBN: 9780470316887. DOI: 10.1002/9780470316887. http://dx.doi.org/10.1002/9780470316887.

[19] Philipp Schiele. 2024. *On Robust Asset Allocation*. Doctoral dissertation. Ludwig-Maximilians-Universität München. DOI: 10.5282/edoc.33594. https://edoc.ub.uni-muenchen.de/33594/.

[20] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. 2018. High-Dimensional Continuous Control Using Generalized Advantage Estimation. (2018). arXiv: 1506.02438 [cs.LG]. https://arxiv.org/abs/1506.02438.

[21] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. (2017). arXiv: 1707.06347 [cs.LG]. https://arxiv.org/abs/1707.06347.

[22] William F. Sharpe. 1964. Capital Asset Prices: A Theory of Market Equilibrium under Conditions of Risk. *The Journal of Finance*, 19, 3, (September 1964), 425. ISSN: 0022-1082. DOI: 10.2307/2977928. http://dx.doi.org/10.2307/2977928.

[23] William F. Sharpe. 1966. Mutual Fund Performance. *The Journal of Business*, 39, S1, (January 1966), 119. ISSN: 1537-5374. DOI: 10.1086/294846. http://dx.doi.org/10.1086/294846.

[24] Satinder P. Singh and Richard S. Sutton. 1996. Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22, 1–3, 123–158. ISSN: 1573-0565. DOI: 10.1007/bf00114726. http://dx.doi.org/10.1007/BF00114726.

[25] Richard S Sutton and Andrew G Barto. 2018. *Reinforcement learning: An introduction*. (Second ed.). MIT press.

[26] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. 1999. Policy Gradient Methods for Reinforcement Learning with Function Approximation. In *Advances in Neural Information Processing Systems*. S. Solla, T. Leen, and K. Müller, (Eds.) Volume 12. MIT Press. https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

[27] Richard S. Sutton. 1988. Learning to predict by the methods of temporal differences. *Machine Learning*, 3, 1, (August 1988), 9–44. ISSN: 1573-0565. DOI: 10.1007/bf00115009. http://dx.doi.org/10.1007/BF00115009.

[28] Ronald J. Williams. 1992. Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning. *Machine Learning*, 8, 229–256. DOI: 10.1007/BF00992696.

[29] Yifan Zhang, Peilin Zhao, Qingyao Wu, Bin Li, Junzhou Huang, and Mingkui Tan. 2020. Cost-Sensitive Portfolio Selection via Deep Reinforcement Learning. *CoRR*, abs/2003.03051. arXiv: 2003.03051. https://arxiv.org/abs/2003.03051.

Appendix A

Full Asset Universe List and PPO Hyperparameter Specification

Table A.1: Asset universe 2019 - 2026

Ticker	Company Name	Ticker	Company Name
MSFT	Microsoft	AAPL	Apple
AMZN	Amazon	GOOG	Alphabet / Google
UNH	UnitedHealth Group	V	Visa
WMT	Walmart	PG	Procter & Gamble
KO	Coca-Cola	JNJ	Johnson & Johnson
PFE	Pfizer	MRK	Merck & Co.
XOM	Exxon Mobil	CVX	Chevron
JPM	JPMorgan Chase	BAC	Bank of America
CAT	Caterpillar	HON	Honeywell
IBM	IBM	INTC	Intel
CSCO	Cisco Systems	MMM	3M
BA	Boeing	DIS	Walt Disney
VZ	Verizon Communications	T	AT&T
CVS	CVS Health	F	Ford Motor Company

The hyperparameters for PPO training are specified in Table A.3. For the monthly specifications, as well as daily rebalancing with transaction costs, the same training parameters were used. For the daily experiment without transaction costs, a separate configuration is used due to different training behavior in this setting.

Table A.2: Asset universe 2007-2015

Ticker	Company Name	Ticker	Company Name
MSFT	Microsoft	AAPL	Apple
AMZN	Amazon	IBM	IBM
INTC	Intel	CSCO	Cisco Systems
WMT	Walmart	PG	Procter & Gamble
KO	Coca-Cola	PEP	PepsiCo
JNJ	Johnson & Johnson	PFE	Pfizer
MRK	Merck & Co.	MCD	McDonald's
CL	Colgate-Palmolive	COST	Costco
HD	Home Depot	DIS	Walt Disney
XOM	Exxon Mobil	CVX	Chevron
JPM	JPMorgan Chase	BAC	Bank of America
CAT	Caterpillar	HON	Honeywell
MMM	3M	BA	Boeing
GE	General Electric	F	Ford Motor Company
VZ	Verizon Communications		

Table A.3: PPO hyperparameter configurations

Hyperparameter	Monthly / Daily with TC	Daily without TC
Policy	MultiInputPolicy	MultiInputPolicy
Number of environments	8	1
Learning rate	0.00005	0.0002
Number of steps	2048	2048
Batch size	64	64
Number of epochs	5	5
Discount factor γ	0.99	0.999
GAE parameter λ_{GAE}	0.95	0.95
Clip range	0.15	0.15
Network architecture	Default	$[256, 256]$ for policy and value function
Normalize advantage	True	True
Entropy coefficient	0.0	0.0
Value function coefficient	0.5	0.5
Maximum gradient norm	0.5	0.5
Use gSDE	False	False
Target KL	None	None
Device	Auto	Auto

Appendix B

2007 - 2015 Regime Data

B.1 Monthly Rebalancing without Transaction Costs

Table B.1: Markowitz estimation window comparison without transaction costs 2007-2015

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD
Markowitz 1 15%	1.059	0.096	0.150	0.633	0.328
Markowitz 1 20%	1.488	0.122	0.200	0.638	0.419
Markowitz 1 30%	3.766	0.218	0.300	0.782	0.534
Markowitz 2 15%	1.175	0.103	0.151	0.676	0.367
Markowitz 2 20%	1.606	0.129	0.200	0.667	0.448
Markowitz 2 30%	2.249	0.161	0.300	0.621	0.582

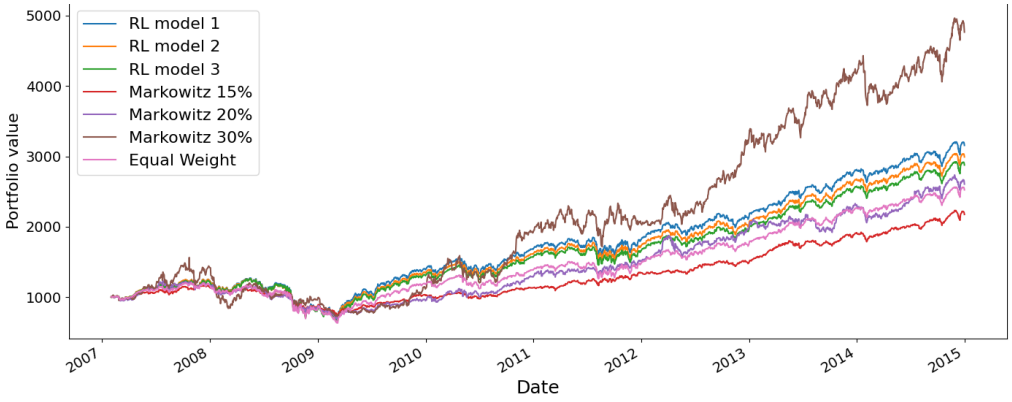


Figure B.1: Monthly rebalancing without transaction costs

The results for this regime are similar to the monthly rebalancing case without transaction costs in the 2019-2026 testing regime. As mentioned in the main text, high volatility Markowitz achieves the best results. This will be even more pronounced throughout the daily rebalancing comparisons.

Table B.2: Monthly rebalancing without transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD
RL Seed 1	2.157	0.156	0.218	0.740	0.452
RL Seed 2	1.992	0.149	0.218	0.710	0.463
RL Seed 3	1.877	0.143	0.219	0.684	0.479
Markowitz 2 15%	1.175	0.103	0.151	0.676	0.367
Markowitz 2 20%	1.606	0.129	0.200	0.667	0.448
Markowitz 1 30%	3.766	0.218	0.300	0.782	0.534
Equal Weight	1.524	0.124	0.209	0.628	0.476

B.2 Daily Rebalancing with Transaction Costs



Figure B.2: Daily rebalancing with transaction costs

Table B.3: Daily rebalancing with transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD	Turnover
RL Seed 1	1.482	0.121	0.210	0.617	0.471	5.887
RL Seed 2	1.443	0.120	0.212	0.604	0.479	5.546
RL Seed 3	1.377	0.116	0.211	0.588	0.480	9.801
Markowitz 15%	0.999	0.091	0.150	0.609	0.330	2.501
Markowitz 20%	1.591	0.128	0.200	0.663	0.405	4.585
Markowitz 30%	4.558	0.242	0.300	0.847	0.449	4.356
Equal Weight	1.549	0.126	0.210	0.632	0.470	1.119

B.3 Daily Rebalancing without Transaction Costs

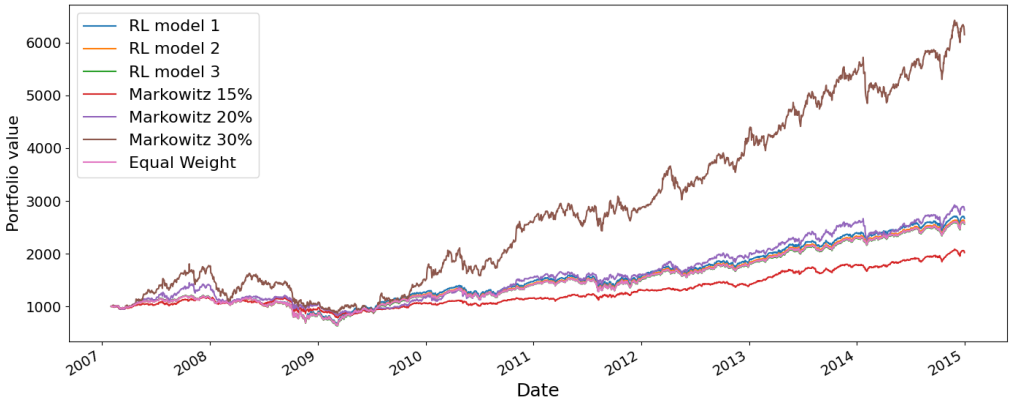


Figure B.3: Daily rebalancing without transaction costs

Table B.4: Daily rebalancing without transaction costs

Method	Cumulative Return	CAGR	Volatility	Sharpe Ratio	Max DD
RL Seed 1	1.669	0.132	0.211	0.657	0.455
RL Seed 2	1.599	0.128	0.210	0.643	0.468
RL Seed 3	1.560	0.126	0.211	0.633	0.478
Markowitz 15%	1.022	0.093	0.150	0.619	0.336
Markowitz 20%	1.828	0.140	0.200	0.719	0.427
Markowitz 30%	5.150	0.258	0.301	0.887	0.518
Equal Weight	1.571	0.127	0.210	0.638	0.469